



Program Studi Teknik Komputer UNSOED

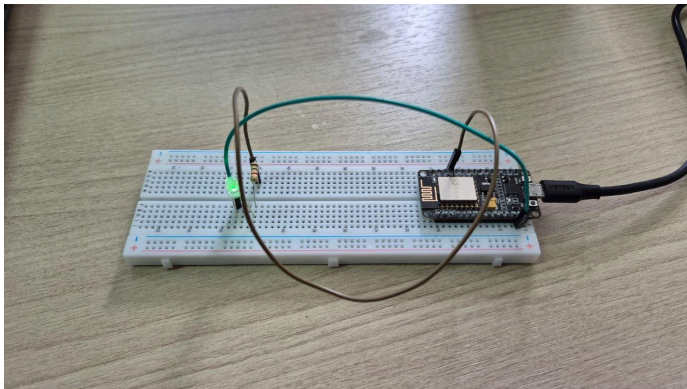
Nama Kuliah (Kode) : Praktikum Sistem Internet of Thing (TK245005)
Tahun / Semester : 2026 / 5
Modul : 2
Nama Asisten / NIM : Athallah Tsany Satriaaji / H1D023013
Nama Praktikan / NIM : Alfian Iskandar Zulkarnain / H1H024034

BUKU CATATAN / LOG PRAKTIKUM

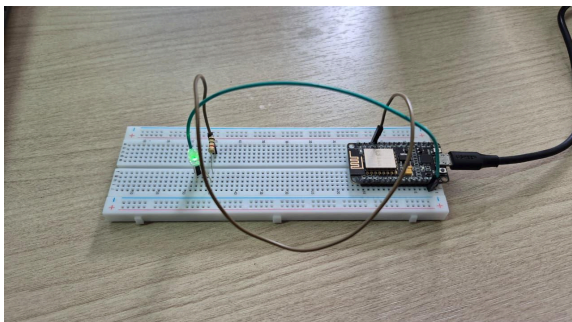
Tujuan Praktikum

1. Memahami konsep dasar jaringan nirkabel (WiFi) pada perangkat IoT berbasis ESP8266.
2. Memahami perbedaan mode Station (STA), Access Point (AP), dan AP+STA pada ESP8266.
3. Mengimplementasikan konfigurasi jaringan untuk menghubungkan ESP8266 ke jaringan WiFi yang sudah tersedia.
4. Mengimplementasikan ESP8266 sebagai Access Point mandiri yang dapat diakses oleh perangkat lain.
5. Mampu membaca dan menganalisis parameter jaringan seperti IP address, MAC address, dan kekuatan sinyal (RSSI).

Gambar Percobaan 2A: Konfigurasi Mode Station (STA)



Gambar Percobaan 2B: Konfigurasi Mode Access Point (AP)



Penjelasan program percobaan 2A: Konfigurasi Mode Station (STA)

```
// include library wifi esp8266
#include <ESP8266WiFi.h>

// =====
// Konfigurasi WiFi
// =====
const char* ssid = "wle";
const char* password = "pppppppp";

// LED indikator
// GPIO 2 pada ESP8266 = D4 pada NodeMCU
#define LED_PIN 2

void setup() {
    // Memulai komunikasi Serial
    Serial.begin(115200);

    // Mengatur pin LED sebagai output
    pinMode(LED_PIN, OUTPUT);

    // LED mati saat awal
    digitalWrite(LED_PIN, LOW);

    // Mengatur ESP8266 sebagai Station
    WiFi.mode(WIFI_STA);

    // Memulai koneksi WiFi
    WiFi.begin(ssid, password);

    // mencetak pesan Menghubungkan ke WiFi ketika mikrokontroler
    mencoba menghubungkan ke wifi
    Serial.println();
    Serial.print("Menghubungkan ke WiFi");

    // Menunggu sampai ESP8266 terhubung
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    // mencetak pesan wifi berhasil terhubung ketika mikrokontroler
    berhasil terhubung ke wifi
```

```

Serial.println();
Serial.println("WiFi berhasil terhubung!");

// menyalakan LED indikator ketika mikrokontroler berhasil
terhubung ke wifi
digitalWrite(LED_PIN, HIGH);

// mencetak informasi jaringan ketika mikrokontroler berhasil
terhubung ke wifi
Serial.print("SSID      : ");
Serial.println(WiFi.SSID());

Serial.print("IP Address  : ");
Serial.println(WiFi.localIP());

Serial.print("MAC Address : ");
Serial.println(WiFi.macAddress());

Serial.print("RSSI      : ");
Serial.print(WiFi.RSSI());
Serial.println(" dBm");
}

void loop() {
    // Cek status koneksi setiap 5 detik
    if (WiFi.status() == WL_CONNECTED) {
        Serial.println("Status: Terhubung");
    } else {
        Serial.println("Status: Terputus");
        digitalWrite(LED_PIN, LOW);
    }
    delay(5000);
}

```

Penjelasan program 2B: Konfigurasi Mode Access Point (AP)

```

// Mengimpor library ESP8266WiFi untuk menggunakan fitur WiFi pada
modul ESP8266
#include <ESP8266WiFi.h>

// Mendefinisikan nama SSID (nama jaringan WiFi) yang akan
dipancarkan oleh Access Point
const char* ap_ssid = "ESP8266 CIIDUK WARRIOR";

```

```

// Mendefinisikan password Access Point (minimal 8 karakter agar
valid)
const char* ap_password = "12345678"; // minimal 8 karakter

// Fungsi setup() dijalankan sekali saat ESP8266 pertama kali
dinyalakan atau di-reset
void setup() {
    // Memulai komunikasi Serial dengan baud rate 115200 untuk
keperluan debug/monitoring
    Serial.begin(115200);

    // Mengatur mode WiFi ESP8266 menjadi Access Point (AP) saja, bukan
Station (STA)
    WiFi.mode(WIFI_AP);
    // Menginisialisasi Access Point dengan SSID dan password yang
sudah didefinisikan
    WiFi.softAP(ap_ssid, ap_password);

    // Membaca alamat IP dari Access Point yang baru dibuat (default:
192.168.4.1)
    IPAddress apIP = WiFi.softAPIP();

    // Mencetak informasi ke Serial Monitor bahwa Access Point sudah
aktif
    Serial.println("Access Point aktif!");
    // Mencetak label "SSID : " ke Serial Monitor tanpa pindah baris
    Serial.print("SSID : ");
    // Mencetak nama SSID Access Point ke Serial Monitor dan pindah
baris
    Serial.println(ap_ssid);
    // Mencetak label "IP Address: " ke Serial Monitor tanpa pindah
baris
    Serial.print("IP Address: ");
    // Mencetak alamat IP Access Point ke Serial Monitor dan pindah
baris
    Serial.println(apIP);
}

// Fungsi loop() dijalankan berulang-ulang selama ESP8266 menyala
void loop() {
    // Membaca jumlah perangkat (client/station) yang sedang terhubung
ke Access Point
    int jumlahClient = WiFi.softAPgetStationNum();

```

```

// Mencetak label "Jumlah perangkat terhubung: " ke Serial Monitor
tanpa pindah baris
Serial.print("Jumlah perangkat terhubung: ");
// Mencetak nilai jumlah client yang terhubung ke Serial Monitor
dan pindah baris
Serial.println(jumlahClient);
// Menunggu selama 5000 milidetik (5 detik) sebelum loop berikutnya
dijalankan
delay(5000);
}

```

Hasil Pengamatan

Percobaan 2A: Konfigurasi Mode Station (STA)

Tabel 1. Pengamatan Koneksi WiFi Mode Station (STA)

No.	Waktu (s)	Status Koneksi	IP Address	MAC Address	RSSI (dBm)	Status LED
1	0	terhubung	10.188.91.7 6	B4:E6:2D:3 A:8F:BC	-42	nyala
2	5	terhubung	10.188.91.7 6	B4:E6:2D:3 A:8F:BC	-42	nyala
3	10	terhubung	10.188.91.7 6	B4:E6:2D:3 A:8F:BC	-42	nyala
4	15	terhubung	10.188.91.7 6	B4:E6:2D:3 A:8F:BC	-42	nyala
5	20	terhubung	10.188.91.7 6	B4:E6:2D:3 A:8F:BC	-42	nyala
6	25	terhubung	10.206.11.7 6	B4:E6:2D:3 A:8F:BC	-39	nyala
7	30	terhubung	10.206.11.7 6	B4:E6:2D:3 A:8F:BC	-39	nyala
8	35	terhubung	10.206.11.7 6	B4:E6:2D:3 A:8F:BC	-39	nyala
9	40	terhubung	10.206.11.7 6	B4:E6:2D:3 A:8F:BC	-39	nyala

10	45	terhubung	10.206.11.7 6	B4:E6:2D:3 A:8F:BC	-39	nyala
----	----	-----------	------------------	-----------------------	-----	-------

Tabel 2. Pengujian Mode Station (STA)

No.	Kondisi Pengujian	SSID	Password	Status Koneksi	Output Serial Monitor	Keterangan
1	Kredensial Benar	wle	pppppppp	terhubung	Status: terhubung	Led menyala
2	Password Salah	wle	11111111	Tidak terhubung	 (hanya titik titik)	Led mati karena password salah
3	SSID Salah	S24	pppppppp	Tidak terhubung	 (hanya titik titik)	Led mati karena SSID salah

Percobaan 2B: Konfigurasi Mode Access Point (AP)

Tabel 3. Pengamatan Access Point (AP)

No.	Parameter	Nilai Konfigurasi	Hasil Pengamatan
1	SSID	ESP32_AccessPoint	ESP32_AccessPoint
2	Password	12345678	Behasil digunakan
3	IP Address AP	192.168.4.1	192.168.4.1
4	Status Access Point	Aktif	Aktif
5	SSID terdeteksi pada client	Ya	Ya
6	Perangkat berhasil terhubung	-	Ya, 1 smartphone

Tabel 4. Pengamatan Jumlah Perangkat Terhubung ke AP

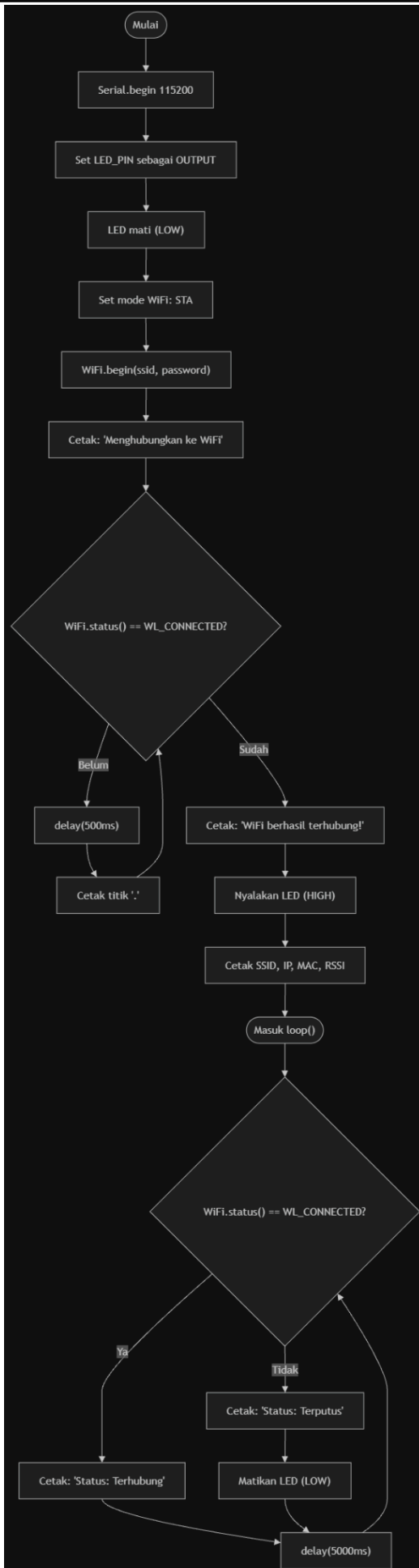
No.	Waktu (s)	Jumlah Client	Perangkat yang terhubung	Keterangan
1	0	1	Smartphone 1	Jumlah perangkat terhubung: 1

2	5	1	Smartphone 1	Jumlah perangkat terhubung: 1
3	10	1	Smartphone 1	Jumlah perangkat terhubung: 1
4	15	2	Smartphone 1, Smartphone 2	Jumlah perangkat terhubung: 2
5	20	2	Smartphone 1, Smartphone 2	Jumlah perangkat terhubung: 2
6	25	2	Smartphone 1, Smartphone 2	Jumlah perangkat terhubung: 2
7	30	3	Smartphone 1, Smartphone 2, Laptop 1	Jumlah perangkat terhubung: 3
8	35	3	Smartphone 1, Smartphone 2, Laptop 1	Jumlah perangkat terhubung: 3
9	40	3	Smartphone 1, Smartphone 2, Laptop 1	Jumlah perangkat terhubung: 3
10	45	3	Smartphone 1, Smartphone 2, Laptop 1	Jumlah perangkat terhubung: 3

Jawaban pertanyaan praktikum

Pertanyaan praktikum percobaan 2A

1. Gambarkan diagram alur (flowchart) proses koneksi ESP32 ke jaringan WiFi pada program di atas!



2. Apa fungsi dari perintah WiFi.mode(WIFI_STA) pada program tersebut?

Jawab : WiFi.mode(WIFI_STA) berfungsi untuk mengatur ESP8266 agar bekerja dalam mode Station (STA), yaitu mode di mana ESP8266 bertindak sebagai client yang terhubung ke jaringan WiFi yang sudah ada (misalnya router/access point), bukan membuat jaringan WiFi sendiri (Access Point). Ini penting agar ESP8266 bisa terhubung ke SSID (ssid) dan password yang telah ditentukan, alih-alih menyediakan hotspot sendiri untuk perangkat lain.

3. Jelaskan apa yang terjadi apabila SSID atau password yang dimasukkan salah!

Jawab : maka mikrokontroler tidak dapat tersambung ke wifi yang akan dituju.

4. Modifikasi program agar ESP32 mencoba menghubungkan ulang (reconnect) secara otomatis apabila koneksi WiFi terputus, dan berikan penjelasan di setiap baris kode yang ditambahkan!

Jawab :

```
// Mengimpor library ESP8266WiFi untuk menggunakan fitur WiFi
pada modul ESP8266
#include <ESP8266WiFi.h>

// Mendefinisikan SSID (nama jaringan WiFi) yang akan dituju
oleh ESP8266 sebagai Station
const char* ssid = "S24";
// Mendefinisikan password dari jaringan WiFi yang akan dimasuki
const char* password = "11111111";

// Mendefinisikan pin GPIO 2 sebagai pin LED indikator status
koneksi WiFi
const int ledPin = 2; // LED indikator status koneksi

// Fungsi setup() dijalankan sekali saat ESP8266 pertama kali
dinyalakan atau di-reset
void setup() {
    // Memulai komunikasi Serial dengan baud rate 115200 untuk
    keperluan debug/monitoring
    Serial.begin(115200);
    // Mengatur pin LED sebagai OUTPUT agar dapat dikontrol
    (nyala/mati)
    pinMode(ledPin, OUTPUT);
    // Mematikan LED saat awal boot (LOW = mati) sebagai kondisi
    awal
    digitalWrite(ledPin, LOW);
```

```

    // Mengatur mode WiFi ESP8266 sebagai Station (STA), yaitu
    perangkat yang terhubung ke router
    WiFi.mode(WIFI_STA);
    // Memulai proses koneksi ke jaringan WiFi menggunakan SSID
    dan password yang didefinisikan
    WiFi.begin(ssid, password);

    // Mencetak teks "Menghubungkan ke WiFi" ke Serial Monitor
    tanpa pindah baris
    Serial.print("Menghubungkan ke WiFi");
    // Loop yang terus berjalan selama status WiFi belum terhubung
    (WL_CONNECTED)
    while (WiFi.status() != WL_CONNECTED) {
        // Menunggu 500 milidetik (0.5 detik) sebelum cek status
        berikutnya
        delay(500);
        // Mencetak titik "." sebagai animasi loading proses koneksi
        Serial.print(".");
    }

    // Mencetak baris kosong (newline) setelah titik-titik loading
    selesai
    Serial.println();
    // Mencetak pesan bahwa WiFi berhasil terhubung
    Serial.println("WiFi berhasil terhubung!");
    // Mencetak label "IP Address : " tanpa pindah baris
    Serial.print("IP Address : ");
    // Mencetak alamat IP lokal yang diberikan oleh router ke
    ESP8266
    Serial.println(WiFi.localIP());

    // Menyalakan LED (HIGH = nyala) sebagai indikator bahwa
    koneksi WiFi berhasil
    digitalWrite(ledPin, HIGH); // nyalakan LED sebagai indikator
    terhubung
}

// Fungsi loop() dijalankan berulang-ulang selama ESP8266
    menyala
void loop() {
    // Memeriksa apakah ESP8266 saat ini masih terhubung ke WiFi
    if (WiFi.status() == WL_CONNECTED) {

```

```

        // Mencetak status "Terhubung" ke Serial Monitor jika
koneksi masih aktif
        Serial.println("Status: Terhubung");
    }
    else {
        // Blok ini dijalankan ketika koneksi WiFi terdeteksi
terputus
        // Mencetak pesan bahwa koneksi terputus dan sedang mencoba
reconnect
        Serial.println("Status: Terputus! Mencoba menghubungkan
ulang (reconnect)...");
        // Mematikan LED karena koneksi WiFi hilang
        digitalWrite(ledPin, LOW); // Matikan LED karena koneksi
hilang

        // Langkah 1: Memutus sisa sesi koneksi yang mungkin masih
menggantung
        WiFi.disconnect();

        // Langkah 2: Memulai kembali proses koneksi ke WiFi dengan
SSID dan password yang sama
        WiFi.begin(ssid, password);

        // Langkah 3: Menunggu proses reconnect dengan batas waktu
maksimal (timeout)
        // Mendeklarasikan variabel counter untuk menghitung
percobaan tunggu
        int waktuTunggu = 0;
        // Loop menunggu koneksi, maksimal 10 kali percobaan (10 x
500ms = 5 detik)
        while (WiFi.status() != WL_CONNECTED && waktuTunggu < 10) {
            // Menunggu 500 milidetik setiap iterasi
            delay(500);
            // Mencetak titik sebagai animasi loading reconnect
            Serial.print(".");
            // Menambah nilai counter setiap iterasi agar tidak
melebihi batas maksimal
            waktuTunggu++;
        }
        // Mencetak newline setelah proses tunggu selesai
        Serial.println();
    }
}

```

```

// Langkah 4: Mengevaluasi apakah proses reconnect berhasil
atau gagal
if (WiFi.status() == WL_CONNECTED) {
    // Jika berhasil terhubung kembali, cetak pesan sukses
    Serial.println("Berhasil terhubung kembali!");
    // Mencetak label "IP Address : " tanpa pindah baris
    Serial.print("IP Address : ");
    // Mencetak alamat IP baru yang diperoleh setelah
reconnect
    Serial.println(WiFi.localIP());
    // Menyalakan LED kembali sebagai tanda koneksi berhasil
    digitalWrite(ledPin, HIGH); // Nyalakan LED kembali
} else {
    // Jika reconnect gagal setelah timeout, cetak pesan
kegagalan
    // ESP8266 akan mencoba lagi pada iterasi loop()
berikutnya
    Serial.println("Gagal menghubungkan ulang. Akan mencoba
lagi pada siklus berikutnya.");
}
}

// Menunggu 5000 milidetik (5 detik) sebelum melakukan
pengecekan status koneksi berikutnya
delay(5000);
}

```

Pertanyaan praktikum percobaan 2B

1. Mengapa alamat IP default Access Point pada ESP8266 umumnya bernilai 192.168.4.1?
Jawab : 192.168.4.1 pada mode Access Point (SoftAP) ESP8266 dipilih oleh pabrikan Espressif Systems untuk menghindari konflik jaringan dengan router rumah yang umumnya menggunakan alamat 192.168.0.1 atau 192.168.1.1.
2. Apa perbedaan mendasar antara mode Station dan mode Access Point pada ESP8266?
Jawab : Pada mode STA, ESP8266 bertindak sebagai klien yang terhubung ke WiFi, mode ini digunakan untuk mengakses internet atau server pada jaringan lokal. Sedangkan mode AP ESP8266 berperan penyedia jaringan (hotspot) yang dapat diakses langsung oleh perangkat lain seperti laptop atau smartphone, tanpa memerlukan router eksternal. Mode ini umum digunakan pada tahap konfigurasi awal perangkat IoT (misalnya untuk mengatur kredensial WiFi melalui halaman web).

3. Jelaskan risiko keamanan apabila password Access Point tidak diberikan atau terlalu sederhana!

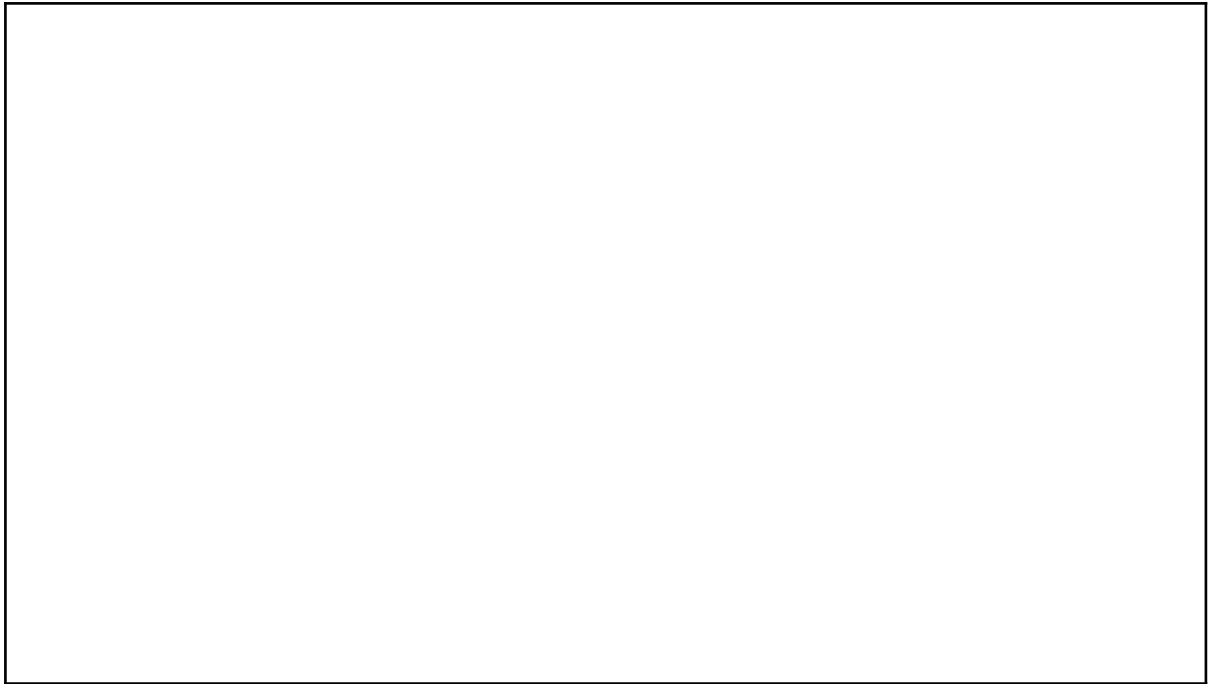
Jawab : jika tidak diberikan password maka siapapun dapat terhubung ke access point dan hal ini akan rentan untuk disadap dan penyerang bisa membanjiri koneksi dengan banyak perangkat palsu yang akan menghabiskan resource ESP8266. Jika password terlalu sederhana maka akan mudah ditebak lewat *brute force* atau *dictionary attack* oleh penyerang.

4. Bagaimana kombinasi mode Station dan Access Point (AP+STA) dapat dimanfaatkan dalam skenario nyata sistem IoT, misalnya pada proses konfigurasi awal perangkat (provisioning)?

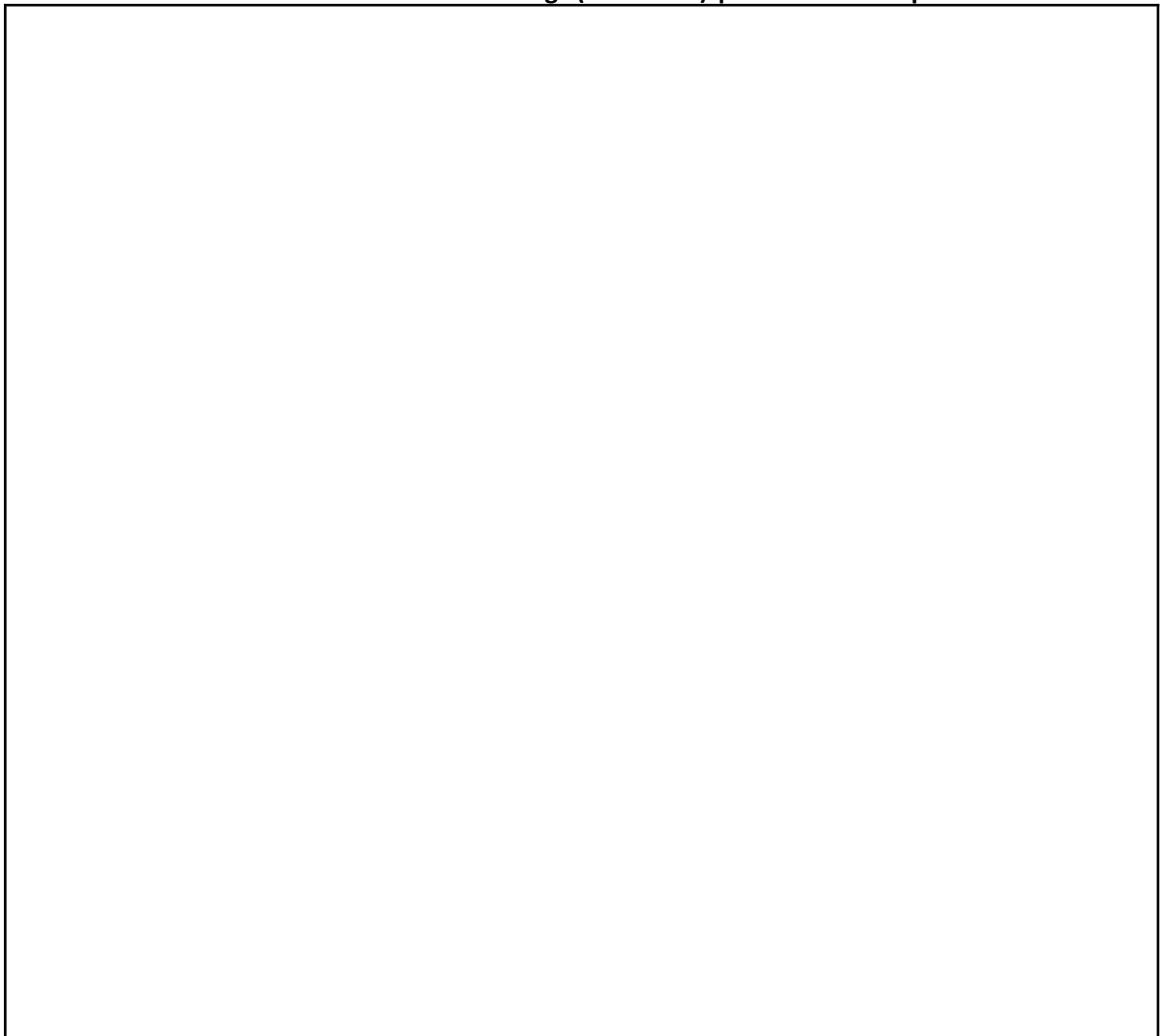
Jawab : AP+STA memungkinkan perangkat IoT jadi Access Point sekaligus Station secara bersamaan. Pada kondisi awal Saat pertama dinyalakan, perangkat belum punya SSID/password WiFi rumah pengguna tersimpan di flash memory. Maka ia otomatis masuk ke mode AP dan memancarkan SSID sendiri, misalnya "ESP-Config-XXXX". Selanjutnya dari HP/laptop, pengguna menghubungkan ke SSID tersebut, lalu membuka browser (biasanya diarahkan otomatis lewat teknik *captive portal*) ke IP 192.168.4.1. Halaman web sederhana muncul, meminta pengguna memasukkan SSID dan password WiFi rumah mereka. Selanjutnya begitu form submit firmware akan mengaktifkan mode AP+STA secara bersamaan, AP tetap aktif agar HP/laptop tidak terputus, mode STA menyala untuk ESP8266 mencoba menyambungkan ke WiFi melalui kredensial yang disubmit. Jika STA berhasil connect (dapat IP dari router rumah), kredensial disimpan permanen di EEPROM/SPIFFS/Preferences, lalu perangkat memberi feedback ke pengguna via halaman web ("Berhasil terhubung!"). Setelah konfirmasi berhasil, perangkat mematikan mode AP (WiFi.mode(WIFI_STA)) dan restart atau lanjut beroperasi hanya sebagai Station — terhubung ke jaringan rumah untuk komunikasi normal (misalnya ke server cloud, MQTT broker, dsb). Jika koneksi STA gagal (password salah, router mati, dll), perangkat tetap mempertahankan mode AP agar pengguna bisa mencoba memasukkan kredensial lagi tanpa perlu me-reset perangkat secara fisik.

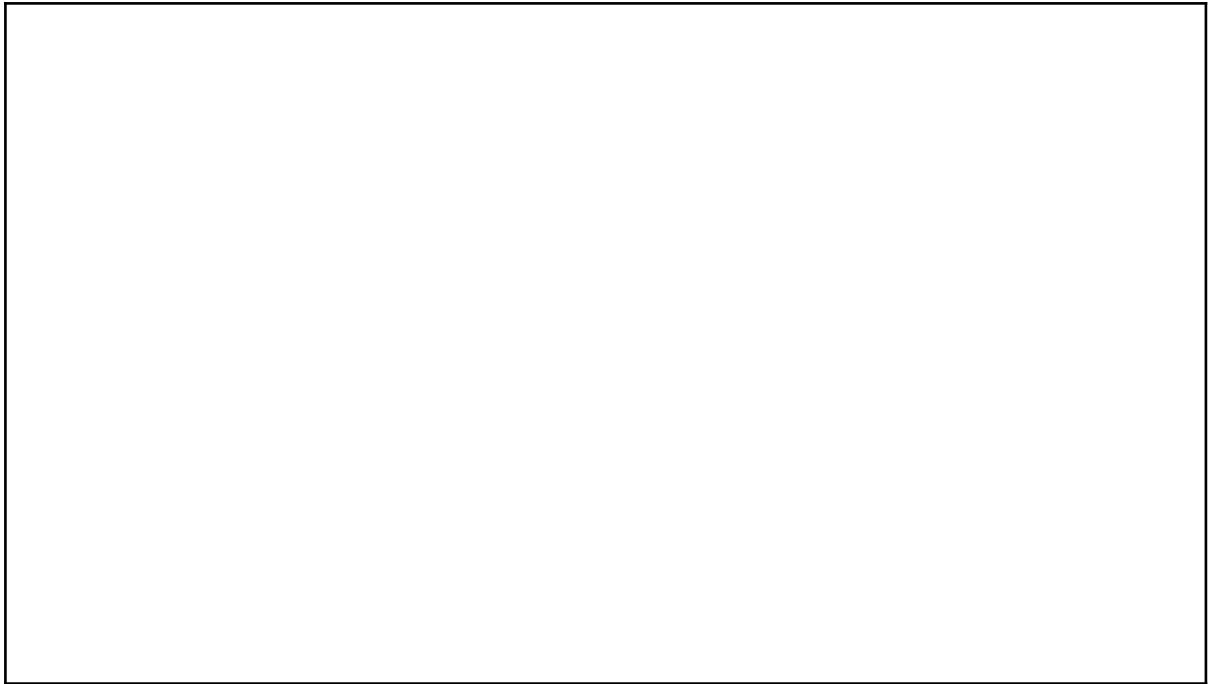
Catatan Kendala

Pada percobaan pertama LED tetap menyala ketika ESP8266 sudah terputus dari WiFi yang harusnya LED mati. Setelah dicari tahu ternyata pin positif LED tertukar salah colok ke pin GPIO. Setelah dibalik LED mati ketika terputus dari WiFi.



Praktikum Internet of Things (TK245005) | Semester Genap





Praktikum Sistem Internet of Things (TK245005) | Semester Genap

